
Kubernetes Labels, Selectors, and Deployment — Teaching Lab

Goal: Learn how to use **labels**, **selectors**, and **deployments** in Kubernetes.

Prerequisites:

- Kubernetes cluster accessible via `kubectl`
 - Basic Linux/terminal skills
-

Step 1: Understand Labels

Definition:

Labels are **key:value pairs** attached to Kubernetes objects (Pods, Nodes, Services, etc.).

- Purpose: Organize, group, and select resources
- Labels **do not have to be unique**
- Example:

metadata:

name: my-nginx

labels:

app: nginx

env: staging

version: v1

Explanation:

Label	Meaning
-------	---------

<code>app=nginx</code>	Application name
------------------------	------------------

`env=staging` Environment type
`g`

`version=v1` Version of the
pod/app

Think of labels as **tags** in Kubernetes.

Step 2: Practice Lab — Add Labels to a Pod

1. Create a file `my-nginx.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-nginx
  labels:
    app: nginx
    env: staging
    version: v1
spec:
  containers:
  - name: nginx
    image: nginx
```

2. Apply the pod:

```
kubectl apply -f my-nginx.yaml
```

3. Check the pod and its labels:

```
kubectl get pods --show-labels
kubectl describe pod my-nginx
```

Step 3: Understand Selectors

Definition:

Selectors are used by Kubernetes to **find resources based on labels**.

- Common uses:
 - Services ✓
 - Deployments/ReplicaSets ✓
 - Network Policies ✓
 - Node scheduling (nodeSelector) ✓

Types of Selectors

1. Equality-based:

selector:

matchLabels:

app: nginx

- Matches resources **with exactly these labels**

2. Set-based:

selector:

matchExpressions:

- key: env

operator: In

values:

- production

- staging

- Matches resources where **env** is **production OR staging**

Use set-based selectors for more **complex filtering**.

Step 4: Lab — Use Selectors with kubectl

- List all pods with `app=nginx`:

```
kubectl get pods -l app=nginx
```

- List pods with `env=staging`:

```
kubectl get pods -l env=staging
```

- Add a label to a running pod:

```
kubectl label pod my-nginx team=dev  
kubectl get pod my-nginx --show-labels
```

Step 5: Deploy a Service Using Labels

1. Create `nginx-service.yaml`:

```
apiVersion: v1  
kind: Service  
metadata:  
  name: nginx-service  
spec:  
  selector:  
    app: nginx  
    env: staging  
  ports:  
    - port: 80
```

targetPort: 80

2. Apply it:

```
kubectl apply -f nginx-service.yaml
```

```
kubectl get svc
```

The service **routes traffic only to pods with matching labels** (**app=nginx** AND **env=staging**).

Step 6: Deploy a Deployment Using Labels

1. Create **nginx-deploy.yaml**:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
        version: v2
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

2. Apply deployment:

```
kubectl apply -f nginx-deploy.yaml  
kubectl get pods -l app=nginx -o wide
```

Explanation:

Section	Meaning
<code>replicas: 3</code>	Run 3 pods
<code>selector.matchLabels</code>	Deployment manages pods with <code>app=nginx</code>
<code>template.metadata.labels</code>	Labels assigned to new pods
<code>version=v2</code>	Can be used for rolling updates

Step 7: Lab — Update Deployment for Rolling Update

1. Update `nginx-deploy.yaml` to `version=v3` in `template.metadata.labels`.
2. Apply again:

```
kubectl apply -f nginx-deploy.yaml  
kubectl get pods -l app=nginx --show-labels
```

- Observe old pods (`version=v2`) are replaced with new pods (`version=v3`).

Labels make **blue/green or canary deployments possible**.

Step 8: Best Practices for Labels

Best Practice	Why
Use consistent label keys	Standardization across teams
Label all resources	Enables visibility & grouping
Include env/team/app labels	Useful for RBAC, cost tracking, network policies
Avoid over-labeling	Simpler labels → faster filtering

Step 9: Quick CLI Reference

Command	Purpose
<code>kubectl get pods -l app=nginx</code>	List pods by label
<code>kubectl get svc -l env=staging</code>	List services by label
<code>kubectl label pod my-nginx team=dev</code>	Add label to running pod
<code>kubectl describe pod my-nginx</code>	Inspect labels & details

✓ Key Takeaways

- Labels = metadata tags for Kubernetes objects
 - Selectors = queries to filter resources by labels
 - Labels are **critical for Services, Deployments, and Network Policies**
 - Proper labeling ensures **visibility, control, and smooth deployments**
-

